

OPEN SOURCE VOICE AI

Advanced RVC Inference

Complete Documentation v2.1.0

[CLI Reference Guide](#) · [Vocoder Reference](#) · [Optimizer Reference](#)

github.com/ArkanDash/Advanced-RVC-Inference

discord.gg/hvmsukmBHE

Table of Contents

Part I — CLI Reference Guide

Installation

Quick Start

infer — Voice Conversion

uvr — Audio Separation

create-dataset — Create Training Data

create-index — Create Model Index

extract — Feature Extraction

preprocess — Data Preprocessing

train — Model Training

create-ref — Create Reference Set

download — Download Models/Audio

serve — Web Interface

info / version / list-models / list-f0-methods

Configuration

Examples & Workflows

Troubleshooting

Part II — Vocoder Reference Guide

Quick Reference Table

Tier 1: Best Quality

Tier 2: Excellent Quality

Tier 3: Very Good Quality

Tier 4: Good Quality

Recommendations

Part III — Optimizer Reference Guide

Quick Reference Table

Tier 1: Best for RVC/Audio Training

Tier 2: Excellent Optimizers

Tier 3: Very Good Optimizers

Tier 4-7: Good to Specialized

Recommendations

Part I — CLI Reference Guide

A comprehensive command-line interface for the Advanced RVC Inference framework. This CLI provides access to all the powerful features of RVC including voice conversion, model training, audio separation, and more — all from the terminal.

Installation

From Source

```
# Clone the repository
git clone https://github.com/ArkanDash/Advanced-RVC-Inference.git
cd Advanced-RVC-Inference

# Install the package
pip install -e .

# Or install with all dependencies
pip install -e .[all]
```

Using the CLI Directly

```
# Run directly with Python
python rvc_cli.py --help

# Or use the wrapper script (Linux/macOS)
./rvc-cli --help

# Make the wrapper executable (Linux/macOS)
chmod +x rvc-cli
```

Quick Start

Basic Voice Conversion

```
# Convert voice with default settings
rvc-cli infer -m mymodel.pth -i input.wav -o output.wav

# Convert with pitch shift (12 semitones = octave up)
rvc-cli infer -m mymodel.pth -i input.wav -p 12

# Convert with custom F0 method
rvc-cli infer -m mymodel.pth -i input.wav --f0_method harvest
```

Audio Separation

```
# Separate vocals from music
rvc-cli uvr -i music.wav

# Separate with specific model
rvc-cli uvr -i music.wav --model MDXNET_Main --output ./separated
```

Launch Web Interface

```
# Start web interface on default port
rvc-cli serve

# Start on custom port with public share
rvc-cli serve --port 8080 --share
```

infer — Voice Conversion

Convert voice in an audio file using an RVC model.

```
rvc-cli infer -m <model> -i <input> [options]
```

Required Arguments

-m, --model	Path to RVC model file (.pth or .onnx)
-i, --input	Path to input audio file

Optional Arguments

-o, --output	Output file path (auto-generated if not specified)
-p, --pitch	Pitch shift in semitones (default: 0)
-f, --format	Output format — wav, mp3, flac, ogg (default: wav)

<code>--index</code>	Path to .index file for better quality
<code>--f0_method</code>	F0 extraction method (default: rmvpe)
<code>--filter_radius</code>	Filter radius for F0 smoothing (default: 3)
<code>--index_rate</code>	Index strength 0.0-1.0 (default: 0.5)
<code>--rms_mix_rate</code>	RMS mix rate (default: 1.0)
<code>--protect</code>	Protect consonants 0.0-1.0 (default: 0.33)
<code>--hop_length</code>	Hop length for processing (default: 64)
<code>--embedder_model</code>	Embedding model (default: hubert_base)
<code>--resample_sr</code>	Resample rate (0 = original, default: 0)
<code>--split_audio</code>	Split audio before processing
<code>--checkpointing</code>	Enable memory checkpointing
<code>--f0_autotune</code>	Enable F0 autotune
<code>--f0_autotune_strength</code>	Autotune strength (default: 1.0)
<code>--formant_shifting</code>	Enable formant shifting
<code>--formant_qfreny</code>	Formant frequency coefficient (default: 0.8)
<code>--formant_timbre</code>	Formant timbre coefficient (default: 0.8)
<code>--clean_audio</code>	Apply audio cleaning
<code>--clean_strength</code>	Cleaning strength (default: 0.7)

EXAMPLE

```
rvc-cli infer -m artist_model.pth -i speech.wav -o converted.wav -p 2 \
  --index_rate 0.75 --f0_method rmvpe --clean_audio
```

uvr — Audio Separation

Separate vocals from instrumentals using UVR5.

```
rvc-cli uvr -i <input> [options]
```

Required Arguments

<code>-i, --input</code>	Path to input audio file
--------------------------	--------------------------

Optional Arguments

<code>-o, --output</code>	Output directory (default: ./audios/uvr)
<code>-f, --format</code>	Output format (default: wav)
<code>--model</code>	Separation model (default: MDXNET_Main)

<code>--karaoke_model</code>	Karaoke model (default: MDX-Version-1)
<code>--reverb_model</code>	Reverb model (default: MDX-Reverb)
<code>--denoise_model</code>	Denoise model (default: Normal)
<code>--sample_rate</code>	Output sample rate (default: 44100)
<code>--shifts</code>	Number of predictions (default: 2)
<code>--batch_size</code>	Batch size (default: 1)
<code>--overlap</code>	Overlap between segments (default: 0.25)
<code>--aggression</code>	Extraction intensity (default: 5)
<code>--hop_length</code>	Hop length (default: 1024)
<code>--window_size</code>	Window size (default: 512)
<code>--enable_tta</code>	Enable test-time augmentation
<code>--enable_denoise</code>	Enable denoising
<code>--separate_backing</code>	Separate backing vocals
<code>--separate_reverb</code>	Separate reverb

AVAILABLE MODELS

MDXNET_Main, MDXNET_9482, HP-Vocal-1, HP-Vocal-2, Inst_HQ_1 through Inst_HQ_5, Kim_Vocal_1, Kim_Vocal_2

EXAMPLE

```
rvc-cli uvr -i song.wav --model HP-Vocal-2 --aggression 10 \
  --enable_denoise --output ./vocals
```

create-dataset — Create Training Data

Create training dataset from YouTube videos or local audio files.

```
rvc-cli create-dataset -u <url> [options]
# or
rvc-cli create-dataset -i <directory> [options]
```

Required Arguments (one of)

<code>-u, --url</code>	YouTube URL (separate multiple with commas)
<code>-i, --input</code>	Input directory with audio files

Optional Arguments

<code>-o, --output</code>	Output directory (default: ./dataset)
---------------------------	---------------------------------------

<code>--sample_rate</code>	Sample rate (default: 48000)
<code>--clean_dataset</code>	Apply data cleaning
<code>--clean_strength</code>	Cleaning strength (default: 0.7)
<code>--separate</code>	Separate vocals (default: True)
<code>--separator_model</code>	Separation model (default: MDXNET_Main)
<code>--skip_start</code>	Seconds to skip at start (default: 0)
<code>--skip_end</code>	Seconds to skip at end (default: 0)

EXAMPLE

```
rvc-cli create-dataset -u "https://youtube.com/watch?v=xxx" \
  --sample_rate 48000 --separate --output ./my_dataset
```

create-index — Create Model Index

Create .index file for voice retrieval.

```
rvc-cli create-index <model_name> [options]
```

<code>model_name</code>	Name of the model (positional)
<code>--version</code>	RVC version — v1 or v2 (default: v2)
<code>--algorithm</code>	Index algorithm — Auto, Faiss, or KMeans (default: Auto)

extract — Feature Extraction

Extract embeddings and F0 from training data.

```
rvc-cli extract <model_name> --sample_rate <rate> [options]
```

<code>model_name</code>	Name of the model (positional)
<code>--sample_rate</code>	Sample rate of input audio (required)
<code>--version</code>	RVC version — v1 or v2 (default: v2)
<code>--f0_method</code>	F0 extraction method (default: rmvpe)
<code>--f0_onnx</code>	Use ONNX F0 predictor
<code>--pitch_guidance</code>	Use pitch guidance (default: True)
<code>--hop_length</code>	Hop length (default: 128)
<code>--cpu_cores</code>	CPU cores (default: 2)

<code>--gpu</code>	GPU index (default: - for CPU)
<code>--embedder_model</code>	Embedder model (default: hubert_base)
<code>--rms_extract</code>	Extract RMS energy

preprocess — Data Preprocessing

Slice and normalize training audio.

```
rvc-cli preprocess <model_name> --sample_rate <rate> [options]
```

<code>model_name</code>	Name of the model (positional)
<code>--sample_rate</code>	Sample rate (required)
<code>--dataset_path</code>	Dataset path (default: ./dataset)
<code>--cpu_cores</code>	CPU cores (default: 2)
<code>--cut_method</code>	Cutting method — Automatic, Simple, or Skip (default: Automatic)
<code>--process_effects</code>	Apply preprocessing effects
<code>--clean_dataset</code>	Clean dataset
<code>--chunk_len</code>	Chunk length for Simple method (default: 3.0)
<code>--overlap_len</code>	Overlap length (default: 0.3)
<code>--normalization</code>	Normalization mode — none, pre, or post (default: none)

train — Model Training

Train a new RVC voice model.

```
rvc-cli train <model_name> [options]
```

<code>model_name</code>	Name of the model (positional)
<code>--version</code>	RVC version — v1 or v2 (default: v2)
<code>--author</code>	Model author name
<code>--epochs</code>	Total training epochs (default: 300)
<code>--batch_size</code>	Batch size (default: 8)
<code>--save_every</code>	Save checkpoint every N epochs (default: 50)
<code>--save_latest</code>	Save only latest checkpoint (default: True)
<code>--save_weights</code>	Save all model weights (default: True)
<code>--gpu</code>	GPU index (default: 0)
<code>--cache_gpu</code>	Cache data in GPU
<code>--pitch_guidance</code>	Use pitch guidance (default: True)

<code>--pretrained_g</code>	Path to pre-trained G weights
<code>--pretrained_d</code>	Path to pre-trained D weights
<code>--vocoder</code>	Vocoder choice (see Part II). Default: HiFi-GAN
<code>--energy</code>	Use RMS energy
<code>--overtrain_detect</code>	Enable overtraining detection
<code>--optimizer</code>	Optimizer choice (see Part III). Default: AdamW
<code>--multiscale_loss</code>	Use multi-scale mel loss
<code>--use_reference</code>	Use custom reference set
<code>--reference_path</code>	Path to reference set
<code>--checkpointing</code>	Enable checkpointing
<code>--cosine_lr</code>	Use CosineAnnealingLR scheduler

create-ref — Create Reference Set

Create reference audio for better inference quality.

```
rvc-cli create-ref <audio_file> [options]
```

<code>audio_file</code>	Path to audio file (positional)
<code>-n, --name</code>	Reference name (default: reference)
<code>--version</code>	RVC version — v1 or v2 (default: v2)
<code>--pitch_guidance</code>	Use pitch guidance (default: True)
<code>--energy</code>	Use RMS energy
<code>--embedder_model</code>	Embedder model (default: hubert_base)
<code>--f0_method</code>	F0 extraction method (default: rmvpe)
<code>--pitch_shift</code>	Pitch shift (default: 0)
<code>--filter_radius</code>	Filter radius (default: 3)
<code>--f0_autotune</code>	Enable F0 autotune
<code>--alpha</code>	Alpha blending (default: 0.5)

download — Download Models/Audio

Download models from HuggingFace or audio from YouTube.

```
rvc-cli download -l <link> [options]
```

<code>-l, --link</code>	Download link (HuggingFace or YouTube URL) (required)
<code>-n, --name</code>	Name to save as

serve — Web Interface

Launch the Gradio web UI.

```
rvc-cli serve [options]
```

--host	Host to bind (default: 0.0.0.0)
--port	Port to bind (default: 7860)
--share	Create public share URL
--easy	Launch Easy GUI (simplified mode)

info / version / list-models / list-f0-methods

rvc-cli info	Show system and environment information (OS, GPU, memory, models)
rvc-cli version	Show version and dependency information
rvc-cli list-models	List installed models in the weights folder
rvc-cli list-f0-methods	Show all available pitch extraction methods

Configuration

Environment Variables

VARIABLE	DESCRIPTION	DEFAULT
ARVC_ASSETS_PATH	Path to assets directory	assets
ARVC_CONFIGS_PATH	Path to configs directory	configs
ARVC_WEIGHTS_PATH	Path to weights directory	assets/weights
ARVC_LOGS_PATH	Path to logs directory	assets/logs

Model and Index Files

Place your model files (`.pth` or `.onnx`) in `arvc/assets/weights/` and index files (`.index`) in `arvc/assets/logs/<model_name>/` .

Examples & Workflows

Complete Training Workflow

```
# 1. Create dataset from YouTube
rvc-cli create-dataset -u "https://youtube.com/watch?v=xxx" \
  --output ./dataset --sample_rate 48000 --separate

# 2. Preprocess data
rvc-cli preprocess mymodel --sample_rate 48000 --cut_method Automatic

# 3. Extract features
rvc-cli extract mymodel --sample_rate 48000 --f0_method rmvpe --gpu 0

# 4. Train model
rvc-cli train mymodel --version v2 --epochs 300 --batch_size 8 --gpu 0

# 5. Create index (after training)
rvc-cli create-index mymodel --version v2
```

Voice Conversion with Different F0 Methods

```
# Using RMVPE (recommended)
rvc-cli infer -m model.pth -i input.wav -o output_rmvpe.wav --f0_method rmvpe

# Using Harvest (faster)
rvc-cli infer -m model.pth -i input.wav -o output_harvest.wav --f0_method harvest

# Using Crepe (most accurate but slow)
rvc-cli infer -m model.pth -i input.wav -o output_crepe.wav --f0_method crepe-medium
```

Batch Processing

```
# Process all audio files in a directory
for file in ./inputs/*.wav; do
  rvc-cli infer -m model.pth -i "$file" -o "./outputs/${basename $file}"
done
```

Troubleshooting

"Model file not found"

Ensure the model path is correct. Check that the model file has `.pth` or `.onnx` extension. Verify file permissions.

"CUDA out of memory"

Reduce batch size: `--batch_size 4` . Enable checkpointing: `--checkpointing` . Use CPU: `--gpu -` .

"Audio format not supported"

Convert audio to WAV first: `ffmpeg -i input.mp3 output.wav` . Supported formats: wav, mp3, flac, ogg, opus, m4a, aac.

"F0 method not available"

Install ONNX runtime for some methods. Some methods require specific embedders.

Getting Help

```
# General help
rvc-cli --help

# Command-specific help
rvc-cli infer --help
rvc-cli uvr --help
rvc-cli train --help
```

Part II — Vocoder Reference Guide

Advanced RVC Inference supports **13 vocoders** for audio synthesis, each with different architectures, strengths, and quality characteristics. This guide provides detailed descriptions, ratings, and recommendations.

Quick Reference

RATING	VOCODER	CATEGORY	KEY FEATURE
★★★★★★	HiFi-GAN (default)	HiFi-GAN	Standard, lightweight & fast
★★★★★★	BigVGAN	Anti-Aliased GAN	SnakeBeta + AMP, highest quality
★★★★★★	Default (HiFi-GAN NSF)	HiFi-GAN	Neural Sine Filter, harmonic injection
★★★★★½	HiFi-GAN-v3	Enhanced HiFi-GAN	SnakeBeta activations, wider dilations
★★★★★½	MRF-HiFi-GAN	Multi-Receptive Field	MRF blocks for richer features
★★★★★½	Vocos	Fourier-based	iSTFT output, lightweight
★★★★★½	RingFormer	Conformer-based	iSTFT + attention, phase prediction
★★★★★	JVSF-HiFi-GAN	Source-Filter	Separate source/filter modeling
★★★★★	RefineGAN	U-Net GAN	Skip connections, parallel ResBlocks
★★★★★	NSF-APNet	Hybrid NSF	All-pass phase correction
★★★★★	FullBand-MRF	Enhanced MRF	Wider dilations, channel attention
★★★★★½	WaveGlow	Flow-based	Invertible convolutions, WaveNet
★★★★★½	PCPH-GAN	Phase-Corrected	PCHIP interpolation, harmonic gen

Tier 1: Best Quality

HiFi-GAN (Default) ★★★★★

Category: HiFi-GAN

Source: [models/generators/hifigan.py](#)

Rating: 5.0/5

HiFi-GAN is the standard and default vocoder. It uses transposed convolution upsampling with weight-normalized residual blocks (Multi-Receptive Field fusion). Each upsampling layer is followed by multiple residual blocks with different kernel sizes and dilation rates, capturing features at multiple temporal scales. Lightweight and fast while still producing high-quality audio output. Works with both pitch-guided and non-pitch-guided training modes.

- Standard transposed convolution upsampling
- Weight-normalized residual blocks with MRF fusion
- Works with and without pitch guidance (f0)
- Lightweight and fast inference

Recommended for: Default choice for all training. Best balance of quality, speed, and compatibility.

BigVGAN ★★★★★

Category: Anti-Aliased GAN

Paper: 2023

Rating: 5.0/5

The highest-quality vocoder available. Introduces Snake activations with Anti-Aliasing (SnakeBeta and AMP blocks) and data-augmented adversarial training. Snake provides a periodic nonlinearity naturally suited for audio signals, while anti-aliased design prevents high-frequency artifacts. Uses kaiser-sinc filters for both upsampling and downsampling, achieving state-of-the-art quality across benchmarks.

- SnakeBeta learnable periodic activations
- Anti-Aliased Multi-Period (AMP) blocks
- Kaiser-sinc filters for upsampling/downsampling
- Superior high-frequency reconstruction

Recommended for: Maximum audio quality. Best for singing voice conversion and high-fidelity speech synthesis.

Default (HiFi-GAN NSF) ★★★★★

Category: HiFi-GAN

Source: `models/generators/nsf_hifigan.py`

Rating: 5.0/5

HiFi-GAN with Neural Sine Filter (NSF). Combines HiFi-GAN's upsampling with a Neural Sine Filter that injects harmonic information directly into each upsampling layer. The NSF source module generates sine waves conditioned on F0, mixed through noise convolution layers. Provides improved pitch accuracy due to explicit harmonic conditioning, but requires pitch guidance (f0) to be enabled.

- Neural Sine Filter for harmonic injection
- Noise convolutions at each upsampling layer
- Improved pitch accuracy from explicit harmonic conditioning
- Supports gradient checkpointing; requires pitch guidance (f0)

Recommended for: Users who need improved pitch accuracy. The explicit harmonic injection helps with singing voice and tonal languages.

Tier 2: Excellent Quality

HiFi-GAN-v3 ★★★★★½

Category: Enhanced HiFi-GAN

Rating: 4.5/5

Enhanced variant incorporating SnakeBeta activations and wider dilation patterns. Unlike original HiFi-GAN which uses LeakyReLU, v3 uses the same Snake activation that made BigVGAN successful, providing periodic nonlinearity better matching audio signal characteristics. Wider dilation captures longer-range temporal dependencies for improved formant transition modeling.

- SnakeBeta learnable activations
- Enhanced residual blocks with wider dilations
- Gradient checkpointing support
- Drop-in upgrade over standard HiFi-GAN

Recommended for: Quality upgrade over Default without the computational cost of BigVGAN.

MRF-HiFi-GAN ★★★★★½

Category: Multi-Receptive Field

Rating: 4.5/5

Replaces standard residual blocks with Multi-Receptive Field (MRF) blocks. Each MRF block contains a sequence of MRFLayers with different dilation stacks, capturing features at multiple temporal scales simultaneously. Particularly effective for speech with complex spectral characteristics. SineGenerator provides harmonic conditioning with 8 harmonics.

- Multi-Receptive Field fusion blocks
- Multiple dilation stacks per block
- SineGenerator with 8 harmonics
- Gradient checkpointing support

Recommended for: Speech with complex spectral characteristics. Good for multi-speaker models.

Vocos ★★★★★½

Category: Fourier-based

Rating: 4.5/5

Fundamentally different approach using inverse Short-Time Fourier Transform (iSTFT) for waveform reconstruction instead of transposed convolution upsampling. Inherently more lightweight because it operates in the frequency domain. The backbone uses 1D convolutions with Snake activations, and the iSTFT head predicts both magnitude and phase for clean waveform output.

- iSTFT-based waveform reconstruction (no transposed convolutions)
- Lightweight and computationally efficient
- Magnitude + phase prediction
- Naturally suited for harmonic signals

Recommended for: Fast inference and lightweight models. Excellent for real-time applications.

RingFormer ★★★★★½

Category: Conformer-based

Rating: 4.5/5

The most architecturally advanced vocoder, using Conformer blocks with both time and frequency attention mechanisms. Uses einops-based rearrange operations for efficient multi-head attention. Output layer uses iSTFT for waveform generation from predicted magnitude and phase. Uses ResBlock_SnakeBeta and SourceModuleHnNSF with 8 harmonics for harmonic conditioning.

- Conformer attention (time + frequency)
- iSTFT output with magnitude + phase prediction
- Most complex architecture; unique 3-output return
- Higher memory usage than other vocoders

Recommended for: Advanced users seeking maximum quality through attention-based processing.

Tier 3: Very Good Quality

JVSF-HiFi-GAN ★★★★★

Category: Source-Filter

Rating: 4.0/5

Joint-Variable Source-Filter explicitly separates voice generation into source (excitation) and filter (vocal tract) components, mirroring the human speech production model. The source module generates learnable harmonic components, while the filter module uses a pyramid of dilated convolutions. This separation allows independent control of pitch-related and timbre-related features.

Recommended for: Voice conversion where preserving speaker identity is critical.

RefineGAN ★★★★★

Category: U-Net GAN

Rating: 4.0/5

U-Net architecture with skip connections. The harmonic downsampling path processes F0 through sine generation and progressive downsampling. The upsampling path uses ParallelResBlocks with three parallel branches (kernel sizes 3, 7, 11) combined through AdaIN noise injection. Skip connections preserve fine spectral details.

Recommended for: High-fidelity audio where spectral detail preservation is important.

NSF-APNet ★★★★★

Category: Hybrid NSF

Rating: 4.0/5

Combines Neural Sine Filter with an All-Pass Network for improved phase modeling. The All-Pass Network applies magnitude-preserving phase corrections at each residual block, addressing inaccurate phase reconstruction common in GAN-based vocoders. Particularly effective in higher frequency ranges where phase errors are most perceptible.

Recommended for: Audio with prominent high-frequency content. Reduces metallic artifacts.

FullBand-MRF ★★★★★

Category: Enhanced MRF

Rating: 4.0/5

Extends MRF-HiFi-GAN with significantly wider dilation patterns (up to rate 15) and channel attention mechanisms. Wider dilations capture longer-range dependencies for better formant transition modeling. Channel attention helps focus on the most informative frequency bands dynamically. Snake activations match the quasi-periodic nature of speech.

Recommended for: Speech with rapid formant transitions or wide pitch ranges.

Tier 4: Good Quality

WaveGlow ★★☆☆½

Category: Flow-based

Rating: 3.5/5

Flow-based vocoder using invertible 1×1 convolutions and WaveNet-style layers with gated activations. Unlike GAN-based vocoders, WaveGlow learns the exact probability distribution of audio waveforms through maximum likelihood estimation with invertible transformations. Provides more stable training but requires more computation.

Recommended for: Users who prefer deterministic models. Avoids adversarial training instability.

PCPH-GAN ★★☆☆½

Category: Phase-Corrected

Rating: 3.5/5

Phase-Corrected Parallel HiFi-GAN uses PCHIP interpolation for F0 upsampling, providing smoother pitch contour transitions. The PCPH generator models pitch harmonics with a specialized source module, while SiLU and SnakeBeta provide smooth nonlinearity. A progressive upsampling path creates a coarse-to-fine synthesis pipeline.

Recommended for: Singing voice conversion where smooth pitch transitions are important.

Recommendations for RVC Training

Beginner

Use **HiFi-GAN** (the default). Lightweight, fast, and works with both pitch-guided and non-pitch-guided training. The best starting point for all users.

Intermediate

Try **BigVGAN** for the highest quality, **Default (HiFi-GAN NSF)** for improved pitch accuracy, or **HiFi-GAN-v3** for a lighter upgrade.

Advanced

Experiment with **RingFormer** for attention-based synthesis, **Vocos** for lightweight inference, or **MRF-HiFi-GAN** for multi-scale feature extraction.

Real-time / Low Latency

Use **Vocos** — its Fourier-based architecture is inherently faster at inference since it avoids expensive transposed convolution upsampling.

Maximum Quality

Use **BigVGAN** — it consistently achieves the highest objective and subjective quality scores across all benchmarks.

TECHNICAL NOTES

- Non-Default/HiFi-GAN vocoders require **v2 + pitch guidance** (enforced by UI)
- Non-Default/HiFi-GAN vocoders use **fp32** at inference (fp16 disabled for stability)
- Vocoder choice is saved in the model checkpoint and used automatically during inference
- Pre-trained weights for non-HiFi-GAN vocoders follow the naming pattern:
`{VocoderName}_f0G48k.pth`
- All vocoders support gradient checkpointing except BigVGAN and WaveGlow
- The vocoder registry is in `arvc/models/generators/__init__.py`

Part III — Optimizer Reference Guide

Advanced RVC Inference supports **43 optimizers** for model training, each with different characteristics, strengths, and use cases. This guide provides detailed descriptions, ratings, and recommendations for RVC/audio model training.

Quick Reference

RATING	OPTIMIZER	CATEGORY	BEST FOR
★★★★★	AdamW	PyTorch Built-in	General-purpose, most reliable
★★★★★	ScheduleFreeAdamW	Schedule-Free	No LR schedule needed
★★★★★	Muon	Second-Order	Large models, fast convergence
★★★★★	Sophia	Second-Order	Large-scale training
★★★★★½	Lion	Sign-Based	Memory-efficient training
★★★★★½	Prodigy	LR-Free	No LR tuning needed
★★★★★½	NAdam	PyTorch Built-in	Faster than standard Adam
★★★★★	RAdam	PyTorch Built-in	Warmup-free training
★★★★★	Adan	Nesterov	Vision and audio tasks
★★★★★	AnyPrecisionAdamW	Mixed-Precision	Bfloat16 training
★★★★★	Ranger21	Combined	RAdam + Lookahead synergy
★★★★★	AdaFactor	Memory-Efficient	Large model training
★★★★★	DAdaptAdam	LR-Free	Automatic LR from gradients
★★★★★	Adam	PyTorch Built-in	Classic adaptive optimizer
★★★★★	PAdam	Partial Adaptive	Adam-SGD interpolation
★★★★★	Apollo	Quasi-Newton	L-BFGS-like convergence
★★★★★½	CAME	Unified	Adam+SGD benefits combined
★★★★★½	NovoGrad	Normalized	Well-conditioned gradients
★★★★★½	ScheduleFreeAdam	Schedule-Free	Adam without LR schedule
★★★★★½	DAdaptAdaGrad	LR-Free	Auto LR with AdaGrad

RATING	OPTIMIZER	CATEGORY	BEST FOR
★★★★	SGD	PyTorch Built-in	Best generalization
★★★★	RMSprop	PyTorch Built-in	RL and recurrent networks
★★★★	AdaBelief / V2	Belief-Based	Better conditioned updates
★★★★	LAMB / LARS	Layer-Adaptive	Large-batch training
★★★½	Adagrad / Adadelata	PyTorch Built-in	Sparse data / no manual LR
★★★½	Adamax / ASGD	PyTorch Built-in	Robust to outliers / convex opt
★★★½	QHAdam / SWATS / Shampoo	Various	Adam-SGD continuum / switching
★★★★	A2Grad / AggMo / PID	Various	Optimal averaging / multi-scale
★★★★	Yogi / Fromage / SM3 / Nero	Various	Stable variance / normalization
★★★★	ScheduleFreeSGD / SOAP	Various	SGD without schedule / 2nd order

Tier 1: Best for RVC/Audio Training

AdamW ★★★★★

Category: PyTorch Built-in

Paper: 2019

Rating: 5.0/5

Adam with decoupled weight decay is the gold standard optimizer for deep learning training. It combines adaptive learning rate with proper L2 regularization by decoupling weight decay from the gradient update. The **default and recommended optimizer** for RVC model training, providing reliable convergence across all architectures, dataset sizes, and training configurations.

- Adaptive learning rates per parameter
- Decoupled weight decay (proper L2 regularization)
- Fused CUDA kernel support for faster training
- Proven track record across all of deep learning

Recommended for: All RVC training scenarios as the default. Works well with LR 1e-4 to 1e-3, batch sizes 4-32, and 100-1000 epochs.

ScheduleFreeAdamW ★★★★★

Category: Schedule-Free

Paper: 2024

Rating: 5.0/5

Eliminates the need for any learning rate scheduling by maintaining dual "z" (lookahead) and "y" (standard AdamW) parameters. The optimizer dynamically adjusts its effective learning rate based on the distance between z and y, providing built-in warmup at the start and natural decay as convergence approaches. Never worry about warmup steps, cosine annealing, or step decay again.

- No learning rate schedule needed whatsoever
- Built-in warmup phase (first ~5% of training)
- Automatic decay as training converges
- Drop-in replacement for AdamW

Recommended for: Users who want to avoid LR schedule tuning. Especially useful with varying dataset sizes.

Muon ★★★★★

Category: Second-Order

Paper: 2025

Rating: 5.0/5

Applies Newton-Schulz iteration to orthogonalize the momentum vector at each step. This normalization provides significantly better conditioning for the optimization landscape, similar to preconditioning in second-order methods but at much lower cost. Demonstrates faster convergence compared to AdamW, particularly in later training stages.

- Momentum orthogonalization via Newton-Schulz iteration
- Better conditioned optimization landscape
- Faster convergence on deep models
- Works well with high learning rates

Recommended for: Advanced users training large RVC models (v2, 48k) wanting faster convergence. Effective with 300+ epoch training runs.

Sophia ★★★★★

Category: Second-Order

Paper: 2023

Rating: 5.0/5

Second-order optimizer using a diagonal Hessian estimate combined with stochastic clipping. Unlike Adam which only uses first-order gradient information, Sophia incorporates curvature information from the Hessian (second derivatives) for more informed update decisions. The diagonal approximation keeps memory manageable while the clipping mechanism prevents excessively large updates in high-curvature directions.

- Diagonal Hessian estimation for curvature awareness
- Stochastic clipping for stability
- Faster convergence than first-order methods
- Memory-efficient diagonal approximation

Recommended for: Users with sufficient GPU memory wanting maximum convergence speed. Best with larger batch sizes (8+) and longer training runs.

Tier 2: Excellent Optimizers

Lion ★★★★★½

Category: Sign-Based

Paper: 2023

Rating: 4.5/5

Discovered through automated program search. Uses the **sign of the momentum** rather than the momentum itself for the update direction. This dramatically simplifies computation and results in ~50% less memory than AdamW (only one state tensor vs. two). Often matches or exceeds AdamW performance, particularly with higher learning rates.

Recommended for: Memory-constrained training or when using higher learning rates than AdamW allows.

Prodigy ★★★★★½

Category: LR-Free

Paper: 2023

Rating: 4.5/5

Automatically determines the optimal learning rate by estimating the distance to the solution (D0) using gradient statistics. You only need to set `d_coef`. The optimizer continuously adapts its effective LR during training, eliminating the most common failure mode — choosing the wrong learning rate.

Recommended for: Users who struggle with LR tuning or need a "set it and forget it" optimizer across different model architectures.

NAdam ★★★★★½

Category: PyTorch Built-in

Paper: 2015

Rating: 4.5/5

Combines Adam's adaptive learning rates with Nesterov accelerated gradient. The Nesterov aspect means the optimizer looks ahead by computing the gradient at the anticipated next position, providing implicit momentum correction that often leads to faster convergence, especially in early training stages.

Recommended for: A slight upgrade over AdamW without the complexity of newer optimizers. Good default alternative.

Tier 3: Very Good Optimizers

RAdam ★★★★★

Category: PyTorch Built-in

Rating: 4.0/5

Rectified Adam dynamically switches between SGD-like updates (when variance is unreliable) and Adam-like updates (when variance becomes trustworthy), eliminating the need for warmup steps.

Recommended for: Short training runs where warmup would consume a significant fraction of total steps.

Adan ★★★★★

Category: Nesterov

Rating: 4.0/5

Introduces a unique third moment that tracks the difference between consecutive gradients, capturing curvature information at first-order cost. Has shown particularly strong results on vision and audio tasks.

Recommended for: Audio/vision training tasks where gradient smoothness matters.

Ranger21 ★★★★★

Category: Combined

Rating: 4.0/5

Synergistically combines RAdam's variance rectification with Lookahead's slow-fast weight synchronization. Every k steps, interpolates between "fast" weights (RAdam) and "slow" weights. Acts as a regularizer that prevents overshooting minima.

Recommended for: A "best of both worlds" optimizer with RAdam's stability and Lookahead's generalization benefits.

AdaFactor ★★★★★

Category: Memory-Efficient

Rating: 4.0/5

Dramatically reduces memory usage by factoring the second-moment estimator into row-wise and column-wise statistics. For a (m, n) parameter matrix, Adam stores $m \times n$ variance values while AdaFactor stores $m + n$. Instrumental in training T5.

Recommended for: Training large RVC models on GPUs with limited memory.

DAdaptAdam ★★★★★

Category: LR-Free

Rating: 4.0/5

Automatically determines the learning rate by estimating the distance to the optimal solution from accumulated gradient statistics. Maintains the Adam update rule but automatically adjusts the effective LR. Set `lr=1.0` and let D-Adapt handle the rest.

Recommended for: Automatic LR tuning while keeping familiar Adam behavior.

PAdam ★★★★★

Category: Partial Adaptive

Rating: 4.0/5

Introduces a `p_partial` parameter that controls how much of the second moment's power to use. At `p_partial=0` it behaves like SGD; at `p_partial=1` it's Adam. Default 0.25 provides a balance retaining some adaptivity while gaining generalization benefits.

Recommended for: A balance between Adam's fast convergence and SGD's good generalization.

Apollo ★★★★★

Category: Quasi-Newton

Rating: 4.0/5

Approximates diagonal Hessian information using the ratio of consecutive gradients, similar to L-BFGS. Provides second-order convergence benefits without full Hessian computation. Starts with Adam-like behavior and progressively incorporates more curvature information.

Recommended for: Quasi-Newton convergence speed without the complexity and memory cost of full second-order methods.

Tier 4–7: Good to Specialized Optimizers

Tier 4: Good (3.5 stars)

OPTIMIZER	KEY FEATURE
CAME	Tracks magnitude and sign consistency; combines Adam per-parameter LR with SGD generalization
NovoGrad	Normalizes gradient by RMS before second moment; better conditioning across layers
ScheduleFreeAdam	Adam variant with built-in warmup/decay; no external LR scheduling needed
DAdaptAdaGrad	Combines AdaGrad's cumulative second moment with D-Adaptation's automatic LR estimation

Tier 5: Solid (3 stars)

OPTIMIZER	KEY FEATURE
SGD	Foundational optimizer; best generalization with proper LR scheduling, but requires more tuning
RMSprop	Moving average of squared gradients; popular in RL and recurrent networks
AdaBelief / V2	Adjusts step size based on "belief" in gradient direction; V2 adds AMSGrad support
LAMB	Per-layer trust ratio; essential for large-batch distributed training
LARS	Per-layer adaptive rate scaling; prevents single-layer gradient domination

Tier 6: Moderate (2.5 stars)

OPTIMIZER	KEY FEATURE
Adagrad	Cumulative squared gradients; larger updates for infrequent parameters; LR can become too small
Adadelta	Addresses Adagrad's decreasing LR by restricting accumulation window
Adamax	Uses infinity norm; more robust to gradient outliers
ASGD	Running average of past parameters; better generalization for convex objectives
DAdaptSGD	SGD + D-Adaptation; SGD generalization without manual LR tuning
QHAdam	Quasi-Hyperbolic Adam; principled continuum between SGD and Adam
SWATS	Starts with Adam, switches to SGD when variance drops; best of both worlds
Shampoo	Layer-wise preconditioning with Kronecker factorization
SOAP	Second-Order Adam-like Preconditioner for distributed training

Tier 7: Specialized (2 stars)

OPTIMIZER	KEY FEATURE
A2Grad	Optimal averaging of iterates with second-order step sizes
AggMo	Multiple momentum buffers at different decay rates; combined update
PID	Proportional-Integral-Derivative control theory applied to gradient descent
Yogi	Controls second moment growth rate; ensures variance is non-decreasing
Fromage	Normalizes updates by Frobenius norm; natural regularization
SM3	Sublinear memory; element-wise maximum of squared gradients
ScheduleFreeSGD	SGD with built-in warmup/decay; no LR scheduling needed
Nero	Weight normalization at each step; natural regularizer

Recommendations for RVC Training

Beginner

Start with **AdamW** (default). Use learning rate 1e-3 with 300 epochs and batch size 8.

Intermediate

Try **ScheduleFreeAdamW** to eliminate LR schedule tuning, or **NAdam** for slightly faster convergence. Drop-in replacements requiring no additional configuration.

Advanced

Experiment with **Sophia** or **Muon** for faster convergence on larger models. **Prodigy** and **DAdaptAdam** eliminate LR tuning entirely.

Memory-Constrained

Use **Lion** (50% less memory than Adam) or **AdaFactor** (sublinear memory scaling).

Large-Batch Training

Use **LAMB** or **LARS** for per-layer adaptive learning rate scaling, preventing gradient explosion in large-batch scenarios.

TECHNICAL NOTES

- All custom optimizers are implemented in `arvc/models/optimizers/`
- The central registry in `__init__.py` maps optimizer names to their classes
- The training engine uses the registry for dynamic optimizer selection
- Each optimizer automatically receives appropriate kwargs (betas, eps, weight_decay) based on its capabilities
- Fused CUDA kernels are automatically enabled when supported (currently only AdamW)
- For optimizers that don't support `betas` or `eps`, these parameters are silently omitted